FACULTY OF ARTS

KU LEUVEN

Computational Linguistics

Lecture 3: Language models and word meaning

Tim Van de Cruys



Tuesday 6 October 2026

Tokenization

- Text is encoded by a computer as a sequence of characters (string)
- Words, spaces, and punctuation are encoded in the same way
- We need to define and mark word boundaries: **tokenization**

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*
- Hyphen? *self-driving, what-do-you-mean-look*

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*
- Hyphen? *self-driving, what-do-you-mean-look*
- And if there is no separation?
  - langeafstandsloper, serotonineheropnameremmer
  - Chinese script: a sequence of characters without whitespace
  - Agglutinating languages: Turkish, Basque, Inuit languages

Tokenization

What is a word?

- Proper names: San Francisco, New York

Tokenization

What is a word?

- Proper names: San Francisco, New York
- Fixed expressions:
  - by and large, once and for all
  - piece of cake, spill the beans
  - et cetera, wishful thinking

Tokenization

What is a word?

- Proper names: San Francisco, New York
- Fixed expressions:
  - by and large, once and for all
  - piece of cake, spill the beans
  - et cetera, wishful thinking
- Modern neural models avoid the question: text is split into frequency-based **subword units** (e.g. byte pair encoding), which may be smaller than words; more on this later in the course

Sentence detection

- Certain punctuation characters are unambiguous (‘?’, ‘!’)
- A dot (‘.’) is very ambiguous
  - *Obama held his final address to the U.N. General Assembly on Wednesday.*
  - *Different genres such as the motet, the cantata, etc. developed throughout the centuries.*
- Context is important to determine where a sentence ends

Language models

- Symbolic models (rule-based) give a hard, binary model of well-formed sentences
- For many natural language processing tasks, a probabilistic model of a language (what is the probability of a string being well-formed?) is more useful
- To specify a correct probability distribution, the probability of all sentences in a language must sum to 1

Uses of language models

- Speech recognition
  - ‘I ate a tomato’ is more likely than ‘eye eight atom eight oh’
- OCR & Handwriting recognition
  - More probable sentences are more likely correct readings
- Machine translation
  - More likely sentences are probably better translations
- Generation
  - More likely sentences are probably better natural language generations
- Context sensitive spelling correction
  - ‘Their is a problem wit this sentence’

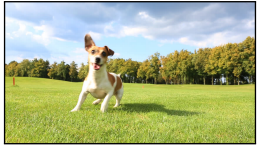
Completion prediction

- Remember the guessing game of lecture 1: a language model predicts the next word in the sentence
  - Please turn off your mobile ____
  - If he loses, he has only himself to ____
- Predictive text input systems can guess what you are typing and provide choices on how to complete it

Statistical language model

- How to compute the probability of a sentence, e.g. *the dog was running across the green field* ?
- We could count the number of times we attest the sentence, and divide by the total number of sentences in our corpus

$$\frac{count(\text{the dog was running across the green field})}{total\ number\ of\ sentences}$$



Statistical language model

- How to compute the probability of a sentence, e.g. *the dog was running across the green field* ?
- We could count the number of times we attest the sentence, and divide by the total number of sentences in our corpus

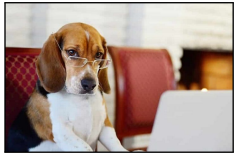
$$\frac{\textit{count}(\text{the dog was running across the green field})}{\textit{total number of sentences}} = \frac{0}{10^{14}} = 0$$



Statistical language model

How to compute $p(\text{the,dog,was,running,across,the,green,field})$?

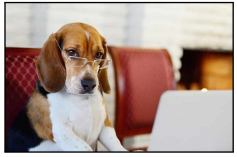
- We compute the joint probability by decomposing into conditional probabilities
- $p(\text{the}) \times p(\text{dog}|\text{the})$
  $\times\ p(\text{was}|\text{the dog})$
  $\times\ p(\text{running}|\text{the dog was})$
  $\times\ p(\text{across}|\text{the dog was running})$
  $\times\ p(\text{the}|\text{the dog was running across})$
  $\times\ p(\text{green}|\text{the dog was running across the})$
  $\times\ p(\text{field}|\text{the dog was running across the green})$



Statistical language model

How to compute p(the,dog,was,running,across,the,green,field) ?

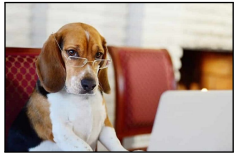
- We compute the joint probability by decomposing into conditional probabilities
- p(the) $\times$ p(dog|the)
  $\times$ p(was|the dog)
  $\times$ p(running|~~the~~ dog was)
  $\times$ p(across|~~the dog~~ was running)
  $\times$ p(the|~~the dog was~~ running across)
  $\times$ p(green|~~the dog was running~~ across the)
  $\times$ p(field|~~the dog was running across~~ the green)
- ... using only a limited context



Statistical language model

How to compute p(the,dog,was,running,across,the,green,field) ?

- We compute the joint probability by decomposing into conditional probabilities
- p(the) $\times$ p(dog|the)
  $\times$ p(was|the dog)
  $\times$ p(running|dog was)
  $\times$ p(across|was running)
  $\times$ p(the|running across)
  $\times$ p(green|across the)
  $\times$ p(field|the green)
- ... using only a limited context



Estimating probabilities

$$p(\text{the}) = \frac{count(the)}{count(*)}$$

$$p(\text{dog}|\text{the}) = \frac{count(the,dog)}{count(the)}$$

$$p(\text{green}|\text{across the}) = \frac{count(across,the,green)}{count(across,the)}$$

N-gram models

- Estimate probability of each word given prior context
- Multiplication of individual probabilites gives sentence probability
- An n-gram model uses only n–1 words of prior context
  - unigram: $p(\text{dog})$
  - bigram: $p(\text{dog} \mid \text{the})$
  - trigram: $p(\text{green} \mid \text{across the})$
  - ... and so on
- N-gram models are based on the **Markov assumption**
  - Future behavior of a dynamical system only depends on its recent history
  - In a kth-order Markov model, the next state only depends on the k most recent states
  - N-gram model is a (n–1)th-order Markov model.

Smoothing

- As with naive Bayes, we're multiplying all individual probabilities to get the global probability
- If one of the probabilities is zero (because of data sparsity), our entire sentence probability will be zero (even if we have other good evidence that the sentence is in fact well-formed)
- We can perform a similar kind of smoothing (add one smoothing) to reassign some probability mass to unseen events
- More complicated smoothing schemes exist
  - Good-Turing
  - Back-off
  - Kneser-Ney

Evaluation

Language models

- As in the previous lecture: train on one part of the corpus, evaluate on **held-out** text
- A good language model assigns a high probability to the held-out sentences
- Standard measure: **perplexity**
  - intuitively: the average number of words the model hesitates between at each position
  - lower is better; a model without any knowledge hesitates between all words of the vocabulary
- **Intrinsic** evaluation: the model is evaluated in itself, not as part of an application such as machine translation or speech recognition (**extrinsic** evaluation)

Inconveniences

- Simple and effective approach that can be applied to large text corpora
- Still, insufficient model of language:
  - Long-distance dependencies
  - Lack of meaning

Long-distance dependencies

- Limited context is unable to predict long-distance dependencies
- Syntactic dependencies
  - *The man that you see standing over there on the corner is tall.*
  - *The men that you see standing over there on the corner are tall.*
- Semantic dependencies
  - *The bird that I was watching during my lunch break flies rapidly.*
  - *The man that I was watching during my lunch break talks rapidly.*
- More complex models of language are needed to handle such dependencies

Lack of meaning

- Discrete approach is often used in natural language processing, and works surprisingly well
- Still, a very poor representation of word meaning
- Atomic word representation

$$p(\text{apple}|\text{eat the})$$

$$p(\text{pear}|\text{eat the})$$

Lack of meaning

- Discrete approach is often used in natural language processing, and works surprisingly well
- Still, a very poor representation of word meaning
- Atomic word representation

$$p(\text{apple}|\text{eat the}) \qquad p(\text{fantastic}|\text{positive})$$

$$p(\text{pear}|\text{eat the}) \qquad p(\text{sensational}|\text{positive})$$

Lack of meaning

- Discrete approach is often used in natural language processing, and works surprisingly well
- Still, a very poor representation of word meaning
- Atomic word representation

$p(\text{apple}|\text{eat the})$ $p(\text{fantastic}|\text{positive})$

$p(\text{pear}|\text{eat the})$ $p(\text{sensational}|\text{positive})$

$[\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,]$



Lack of meaning

- Discrete approach is often used in natural language processing, and works surprisingly well
- Still, a very poor representation of word meaning
- Atomic word representation

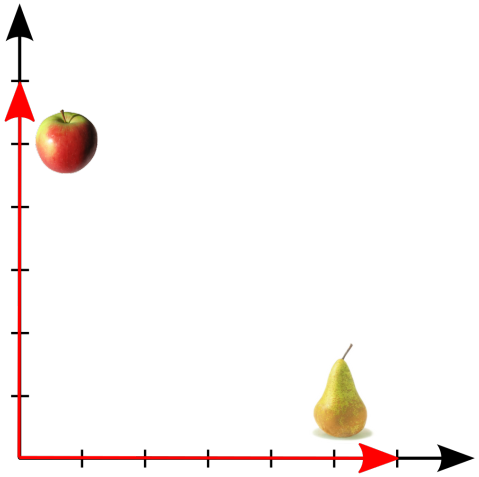
$p(\text{apple}|\text{eat the})$ $p(\text{fantastic}|\text{positive})$

$p(\text{pear}|\text{eat the})$ $p(\text{sensational}|\text{positive})$

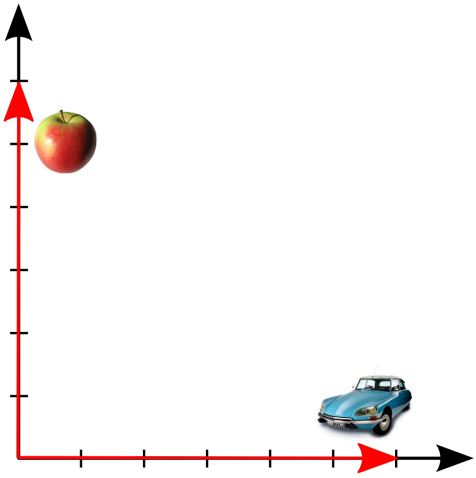
$[\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,]$ 

$[\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,]$ 

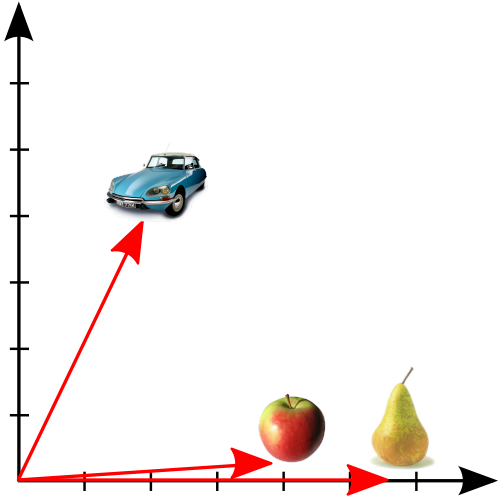
Vectors



Vectors



Vectors



Introduction

- Instead, we want words to have a rich feature structure, representing their properties

	abstract	artefact	edible	ethyl 2-methylbutyrate
apple	−	−	+	+
pear	−	−	+	−
car	−	+	−	−

- Too much work to create feature structures by hand
- We want the feature vectors to be created automatically

Distributional similarity

- Semantic similarity is based on the DISTRIBUTIONAL HYPOTHESIS
  - Take a word and its contexts:
    - tasty *sooluceps*
    - sweet *sooluceps*
    - stale *sooluceps*
    - freshly baked *sooluceps*
- By looking at a word's context, one can infer its meaning

Distributional similarity

- Semantic similarity is based on the DISTRIBUTIONAL HYPOTHESIS
  - Take a word and its contexts:
    - tasty *sooluceps*
    - sweet *sooluceps*
    - stale *sooluceps*
    - freshly baked *sooluceps*

    $\Rightarrow$ **food**
- By looking at a word's context, one can infer its meaning

Distributional similarity

- Semantic similarity is based on the DISTRIBUTIONAL HYPOTHESIS

  - Take a word and its contexts:
    - tasty *sooluceps*
    - sweet *sooluceps*
    - stale *sooluceps*
    - freshly baked *sooluceps*



- By looking at a word's context, one can infer its meaning

Matrix

- captures co-occurrence frequencies of two entities

	red	tasty	fast	second-hand
raspberry	2	1	0	0
strawberry	2	2	0	0
car	1	0	1	2
truck	1	0	1	1

Matrix

- captures co-occurrence frequencies of two entities

	red	tasty	fast	second-hand
raspberry	7	9	0	0
strawberry	12	6	0	0
car	7	0	8	4
truck	2	0	3	4

Matrix

- captures co-occurrence frequencies of two entities

	red	tasty	fast	second-hand
raspberry	56	98	0	0
strawberry	44	34	0	0
car	23	0	31	39
truck	4	0	18	29

Matrix

- captures co-occurrence frequencies of two entities

	red	tasty	fast	second-hand
raspberry	728	592	1	0
strawberry	1035	437	0	2
car	392	0	487	370
truck	104	0	393	293

Neural network: basics

- A single neuron works like logistic regression
- If you know logistic regression (or maximum entropy models), you're familiar with the building blocks of neural networks
- Reminder: logistic regression model yields probability of an instance belonging to a particular class based on the instance's features weighted by the model's parameters

Logistic regression

- Spam classification: determine whether incoming mail is spam
- We take word frequency as features

feature f_i	weight w_i
pharmacy	0.4
viagra	1.2
best	0.2
offer	0.2
request	−0.8
sending	−1.7
bias	0.1

Logistic regression

- Spam classification: determine whether incoming mail is spam
- We take word frequency as features

x_1: "**Pharmacy** online: **viagra best offer**!"

feature f_i	weight w_i
pharmacy	0.4
viagra	1.2
best	0.2
offer	0.2
request	−0.8
sending	−1.7
bias	0.1

- $score(x_1) = 0.4 \times 1 + 1.2 \times 1 + 0.2 \times 1 + 0.2 \times 1 + (-0.8) \times 0 + (-1.7) \times 0 + 0.1 = 2.1$
-

Logistic regression

- Spam classification: determine whether incoming mail is spam
- We take word frequency as features

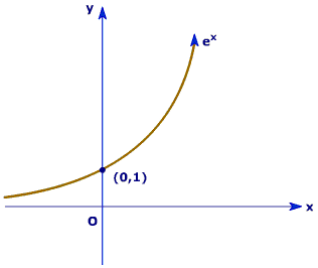
x_2: "Following your **request**, I am **sending** you our **best offer**"

feature f_i	weight w_i
pharmacy	0.4
viagra	1.2
best	0.2
offer	0.2
request	−0.8
sending	−1.7
bias	0.1

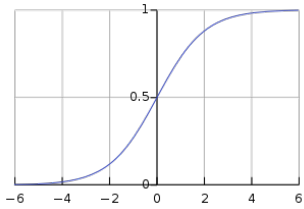
- $score(x_1) = 0.4 \times 1 + 1.2 \times 1 + 0.2 \times 1 + 0.2 \times 1 + (-0.8) \times 0 + (-1.7) \times 0 + 0.1 = 2.1$
- $score(x_2) = 0.4 \times 0 + 1.2 \times 0 + 0.2 \times 1 + 0.2 \times 1 + (-0.8) \times 1 + (-1.7) \times 1 + 0.1 = -2.0$

Logistic regression

- Linear scores (range $-\infty$ to ∞) difficult to interpret, we'd rather have probabilities
- Linear scores are transformed using (non-linear) *logistic function*



exponential function (e^x)

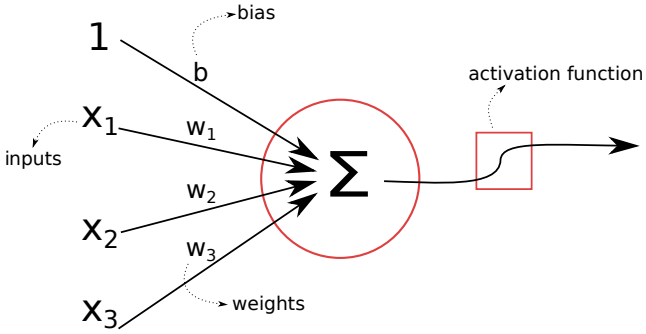


logistic function ($\frac{1}{1+e^{-x}}$)

- $f(score(x1)) = \frac{1}{1+e^{-2.1}} = .89$
- $f(score(x2)) = \frac{1}{1+e^{2.0}} = .12$

Neuron = logistic regression

- These are the exact computations of a single neuron

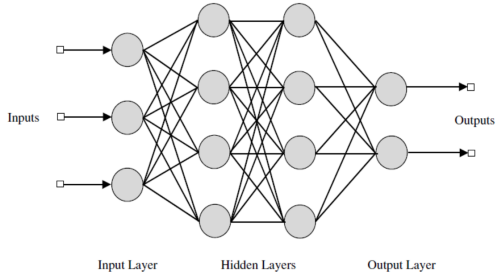


- We can feed an input vector to a bunch of logistic regression functions and get an output vector
- which can be fed to another layer of logistic regression functions

Feed-forward

Architecture

Multi-layer perceptron with 2 hidden layers



$$NN_{MLP2}(\mathbf{x}) = \mathbf{y} \quad (1)$$

$$\mathbf{h}^1 = g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \quad (2)$$

$$\mathbf{h}^2 = g(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \quad (3)$$

$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3 \quad (4)$$

$\mathbf{x}$: vector of size $d_{in} = 3$

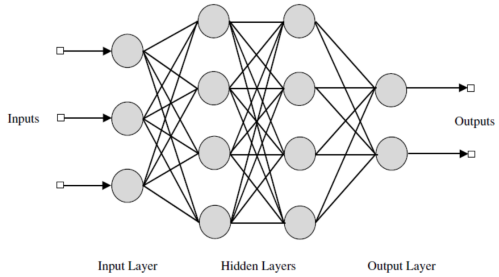
$\mathbf{y}$: vector of size $d_{out} = 2$

$\mathbf{h}^1, \mathbf{h}^2$: vectors of size $d_{hidden} = 4$

Feed-forward

Architecture

Multi-layer perceptron with 2 hidden layers



$$NN_{MLP2}(\mathbf{x}) = \mathbf{y} \quad (1)$$

$$\mathbf{h}^1 = g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \quad (2)$$

$$\mathbf{h}^2 = g(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \quad (3)$$

$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3 \quad (4)$$

$\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3$: matrices of size $[3 \times 4], [4 \times 4], [4 \times 2]$

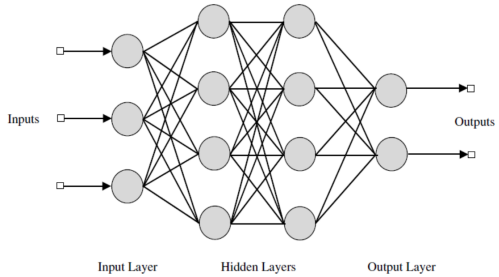
$\mathbf{b}^1, \mathbf{b}^2$: 'bias' vectors of size $d_{hidden} = 4$

$g(\cdot)$: non-linear activation function (elementwise)

Feed-forward

Architecture

Multi-layer perceptron with 2 hidden layers



$$NN_{MLP2}(\mathbf{x}) = \mathbf{y} \quad (1)$$

$$\mathbf{h}^1 = g(\mathbf{x}\mathbf{W}^1 + \mathbf{b}^1) \quad (2)$$

$$\mathbf{h}^2 = g(\mathbf{h}^1\mathbf{W}^2 + \mathbf{b}^2) \quad (3)$$

$$\mathbf{y} = \mathbf{h}^2\mathbf{W}^3 \quad (4)$$

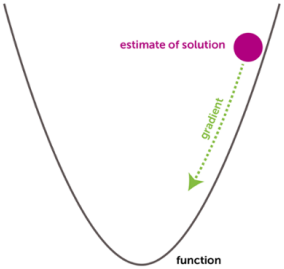
$\mathbf{W}^1, \mathbf{W}^2, \mathbf{W}^3, \mathbf{b}^1, \mathbf{b}^2$ = parameters of the network (θ)

Use of multiple hidden layers: **deep learning**

Training

Loss function

- $L(\hat{y}, y)$ - the loss of predicting $\hat{y}$ when true output is y
- Set parameters θ in order to minimize loss across different training examples
- Compute **gradient** of parameters with regard to loss function to find minimum, take steps in right direction



Training

Loss function

- Categorical cross-entropy loss (negative log-likelihood)
  - Measure difference between true class distribution y and predicted class distribution $\hat{y}$
  - Use with softmax output (network output normalized to probabilities)

Training

Stochastic gradient descent

- Goal: minimize total loss $\sum_{i=1}^{n} L(f(\mathbf{x_i}; \theta), \mathbf{y_i})$
- Estimating gradient over entire training set before taking step is computationally heavy
- Compute gradient for small batch of samples from training set
  $\rightarrow$ estimate of gradient: **stochastic**
- Learning rate λ: size of step in right direction
- Improvements: momentum, adaptive learning rate

Training

Stochastic gradient descent

- Size mini-batch: balance between better estimate and faster convergence
- Gradients over different parameters (weight matrices, bias terms, embeddings, ...) efficiently calculated using **backpropagation** algorithm
- No need to carry out derivations yourself: automatic tools for gradient computation using computational graphs

Word embeddings

- Each word i is represented by a small, dense vector $\mathbf{v}_i \in \mathbb{R}^d$
- d is typically in the range 50–1000
- Matrix of size V (vocabulary size) $\times$ d (embedding size)
- words are 'embedded' in a real-valued, low-dimensional space
- Similar words have similar embeddings

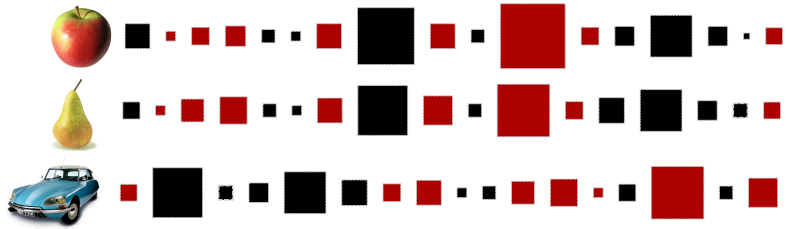
Word embeddings

- Each word i is represented by a small, dense vector $\mathbf{v}_i \in \mathbb{R}^d$
- d is typically in the range 50–1000
- Matrix of size V (vocabulary size) $\times$ d (embedding size)
- words are 'embedded' in a real-valued, low-dimensional space
- Similar words have similar embeddings

	d_1	d_2	d_3	...
apple	−2.34	−1.01	0.33	
pear	−2.28	−1.20	0.11	
car	−0.20	1.02	2.44	
...				

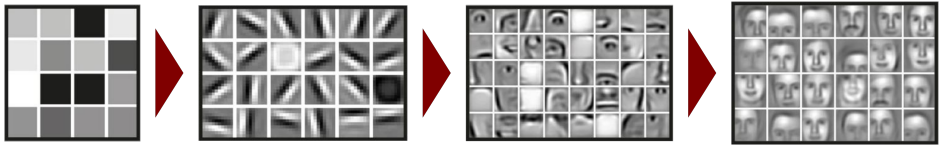
Word embeddings

- Each word i is represented by a small, dense vector $\mathbf{v}_i \in \mathbb{R}^d$
- d is typically in the range 50–1000
- Matrix of size V (vocabulary size) $\times$ d (embedding size)
- words are 'embedded' in a real-valued, low-dimensional space
- Similar words have similar embeddings



Neural word embeddings

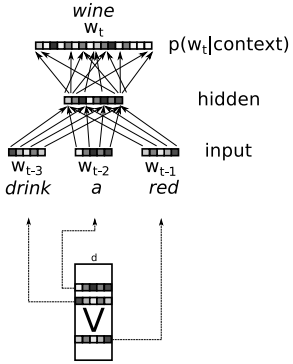
- Word embeddings have been around for quite some time
- The term ‘embedding’ was coined within the neural network community, along with new methods to learn them



- Idea: Let’s allocate a number of parameters for each word and allow the neural network to automatically learn what the useful values should be
- Prediction-based: learn to predict the next word

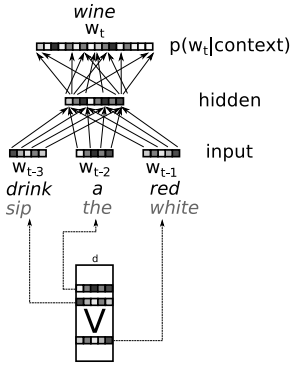
Embeddings through language modeling

- Predict the next word in a sequence, based on the previous word
- One non-linear hidden layer, one softmax layer for classification
- Choose parameters that optimize probability of correct word



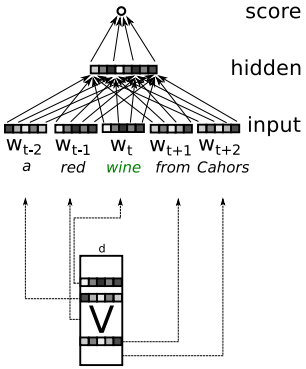
Embeddings through language modeling

- Predict the next word in a sequence, based on the previous word
- One non-linear hidden layer, one softmax layer for classification
- Choose parameters that optimize probability of correct word



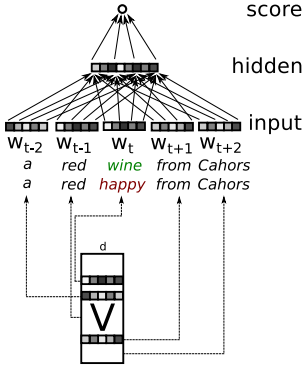
Embeddings through error detection

- Take a correct sentence and create a corrupted counterpart
- Train the network to assign a higher score to the correct version of each sentence



Embeddings through error detection

- Take a correct sentence and create a corrupted counterpart
- Train the network to assign a higher score to the correct version of each sentence

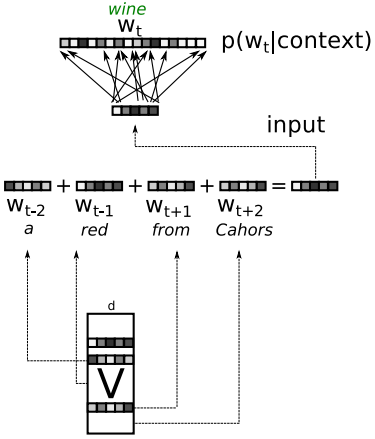


Word2vec

- Neural network approaches work well, but large number of parameters makes them computationally heavy
- Popular, light-weight approach with less parameters: **word2vec**
- No hidden layer, only softmax classifier
- Two different models
  - Continuous bag of words (CBOW): predict current word based on surrounding words
  - Skip-gram: predict surrounding words based on context words

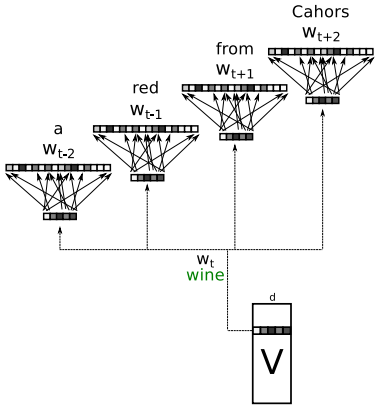
CBOW

- Current word w_t is predicted from context words
- Prediction is made from the sum of context embeddings



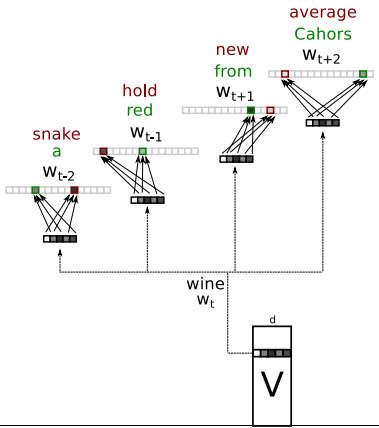
Skip-gram

- Each context word is predicted from current word
- Parameters for each softmax classifier are shared



Negative sampling

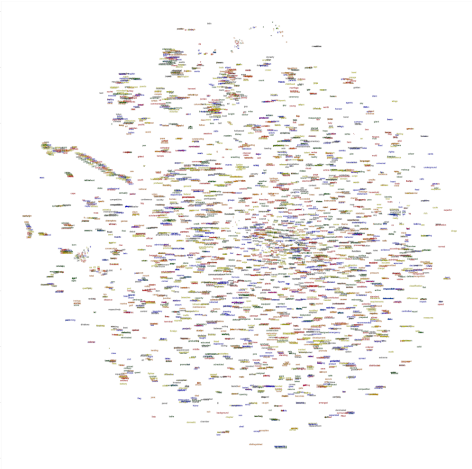
- Computation of full softmax classifier is still rather expensive
- Only compute score for correct context, and a number of wrong contexts
- Maximize correct contexts and minimize wrong ones



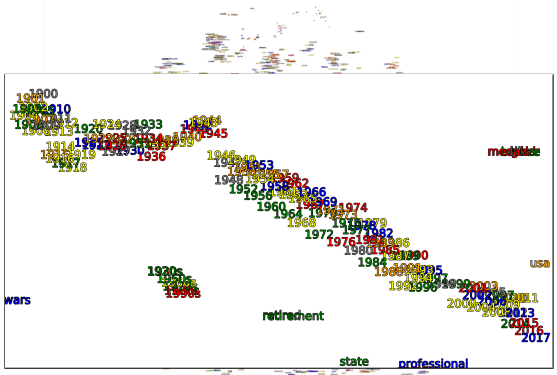
Word similarity

belgium	**jesus**	**red**	**painting**
netherlands	christ	yellow	sculpture
luxembourg	saviour	white	watercolor
france	resurrection	pink	printmaking
switzerland	redeemer	purple	portraiture
brussels	prophet	green	engraving
austria	apostles	beige	painter
flanders	god	black	watercolors
germany	crucifixion	orange	canvases
finland	saints	blueish	painted
italy	angels	multicolored	portrait

Word similarity



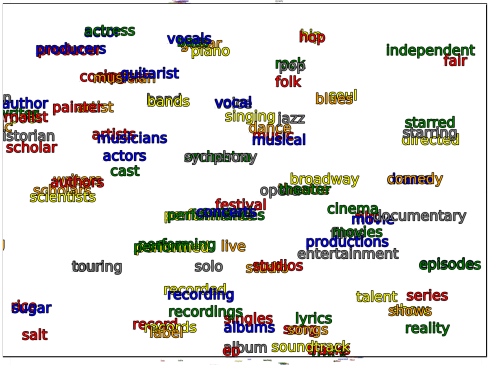
Word similarity



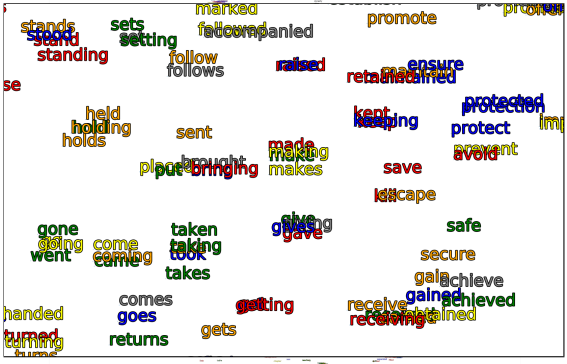
Word similarity



Word similarity



Word similarity



Computing analogies

- Questions in the form:

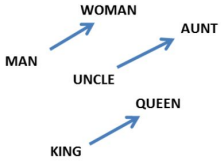
$$a \text{ is to } b \text{ as } c \text{ is to } d$$

- The system is given a, b, and c; it needs to compute d. For example:

apple is to *apples* as *car* is to *?*
man is to *woman* as *king* is to *?*

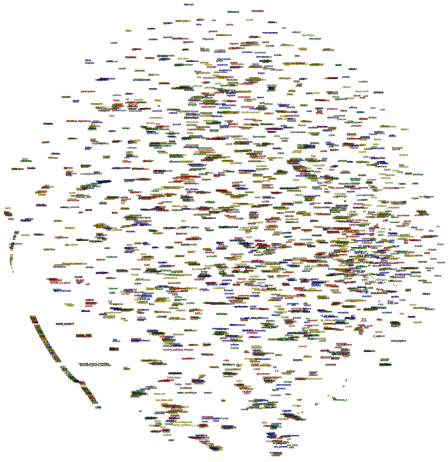
Computing analogies

- **task**: *a* is to *b* as *c* is to *d*
- **idea**: direction of the relation should remain the same

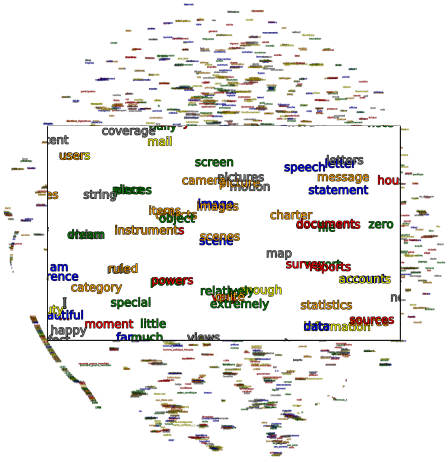


$$
\begin{aligned}
a - b &\approx c - d \\
man - woman &\approx king - queen \\
queen &\approx king - man + woman \\
d_w &= \underset{d'_w \in V}{\text{argmax}}\ cos(d', c - a + b)
\end{aligned}
$$

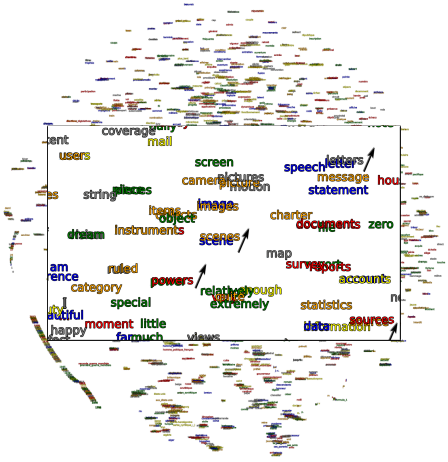
Computing analogies



Computing analogies



Computing analogies



Computing analogies

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

from Mikolov et al. (2013)

Prediction-based vs. count-based

- **Count-based**: extract co-occurrence frequencies from corpus
- **Prediction-based**: induce parameters based on prediction of correct word
- Initially seen as two different methods, where prediction-based approach works better
- Later, emerging evidence and proof that they are actually equivalent or quite similar

Word embeddings in practice

- Word embeddings are often used for pretraining
- Unsupervised: only requires plain text, so can be trained on a lot of data
- Fast algorithms available
- It helps a model start from an informed position
- Model is initialized with pretrained word embeddings, and then finetuned depending on task